UNIT-3

MAP REDUCE

Writing Map Reduce Programs: A Weather Dataset, Filtering Streams using Bloom filters, Understanding Hadoop API for Map Reduce Framework (Old and New), Hadoop Streaming, Basic programs of Hadoop Map Reduce Types and Formats, Anatomy of a Map Reduce Job run, Failures, Map Reduce: Driver code, Mapper code, Reducer code, Record Reader, Combiner, Partitioner.

3.0. INTRODUCTION TO MAPREDUCE

MapReduce is a framework using which the user can write applications to process huge amounts of data, in parallel, on large clusters of commodity hardware in a reliable manner.

Definition: MapReduce is a processing technique and a program model for distributed computing based on java. The MapReduce algorithm contains two important tasks, namely Map and Reduce. Map takes a set of data and converts it into another set of data, where individual elements are broken down into tuples (key/value pairs). Secondly, reduce task, which takes the output from a map as an input and combines those data tuples into a smaller set of tuples. As the sequence of the name MapReduce implies, the reduce task is always performed after the map job.

3.1.1. TERMINOLOGY

- **PayLoad** - Applications implement the Map and the Reduce functions, and form the core of the job.
- **Mapper** - Mapper maps the input key/value pairs to a set of intermediate key/value pair.
- **NamedNode** - Node that manages the Hadoop Distributed File System (HDFS).
- **DataNode** - Node where data is presented in advance before any processing takes place.
- **MasterNode** - Node where JobTracker runs and which accepts job requests from clients.
- **SlaveNode** - Node where Map and Reduce program runs.
- **JobTracker** - Schedules jobs and tracks the assign jobs to Task tracker.
- **Task Tracker** - Tracks the task and reports status to JobTracker.
- **Job** - A program is an execution of a Mapper and Reducer across a dataset.
- **Task** - An execution of a Mapper or a Reducer on a slice of data.
- **Task Attempt** - A particular instance of an attempt to execute a task on a SlaveNode.

3.1.2. ADVANTAGE OF MAPREDUCE: The major advantage of Map Reduce is that it is easy to scale data processing over multiple computing nodes. Under the Map Reduce model, the data processing primitives are called mappers and reducers. Decomposing a data processing application into mappers and reducers is sometimes nontrivial. But, once we write an application in the Map Reduce form, scaling the application to run over hundreds, thousands, or even tens of thousands of machines in a cluster is merely a configuration change. This simple scalability is what has attracted many programmers to use the Map Reduce model.

3.1. WRITING MAP REDUCE PROGRAMS: A WEATHER DATASET

A Map-Reduce program can process large-scale weather datasets to identify temperature extremes. By Hadoop's parallel processing capabilities, program efficiently pinpoints hot and cold days an essential step for climate trend analysis, anomaly detection and building reliable forecasting systems.

Problem Statement: Analyse semi-structured weather data collected by sensors globally, will focus on temperature values (maximum and minimum) and identify **hot days** (temperature > 30°C) and **cold days** (temperature < 15°C) using Map-Reduce.

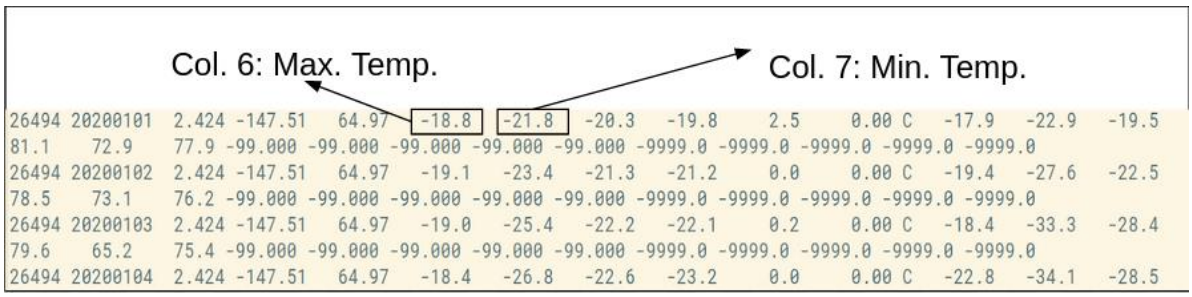
Dataset Overview: We used weather data from the NCEI, available in line-based ASCII text format. Each file contains fields like Date, Latitude, Longitude, Max Temp and Min Temp.

FileName: CRND0103-2020-AK_Fairbanks_11_NE.txt.

Step-by-Step Implementation:

Step 1: Understand Data Format

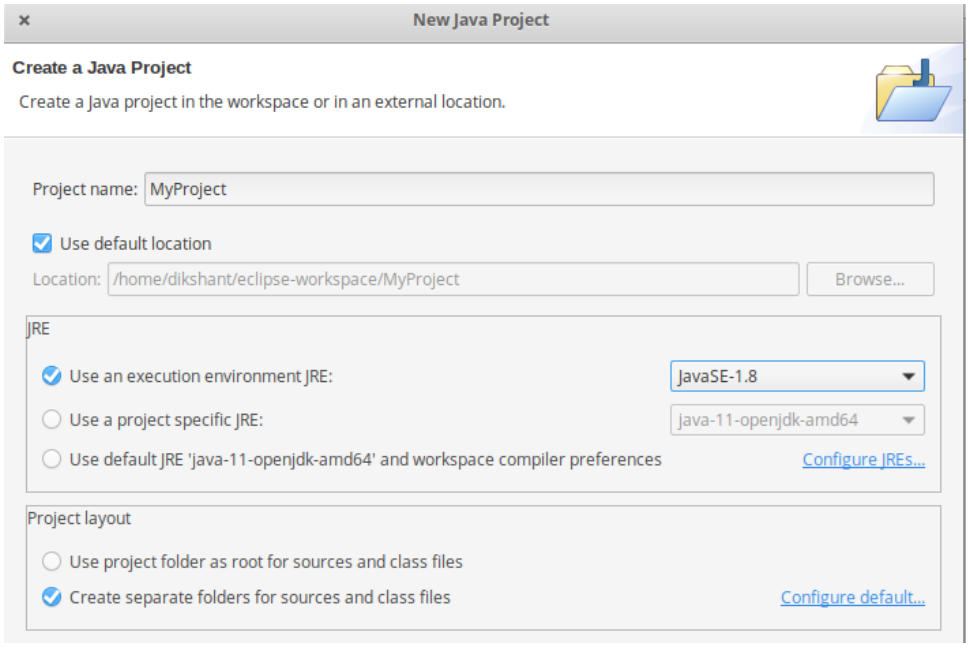
Below is the example of our dataset where column 6 and column 7 is showing Maximum and Minimum temperature, respectively.



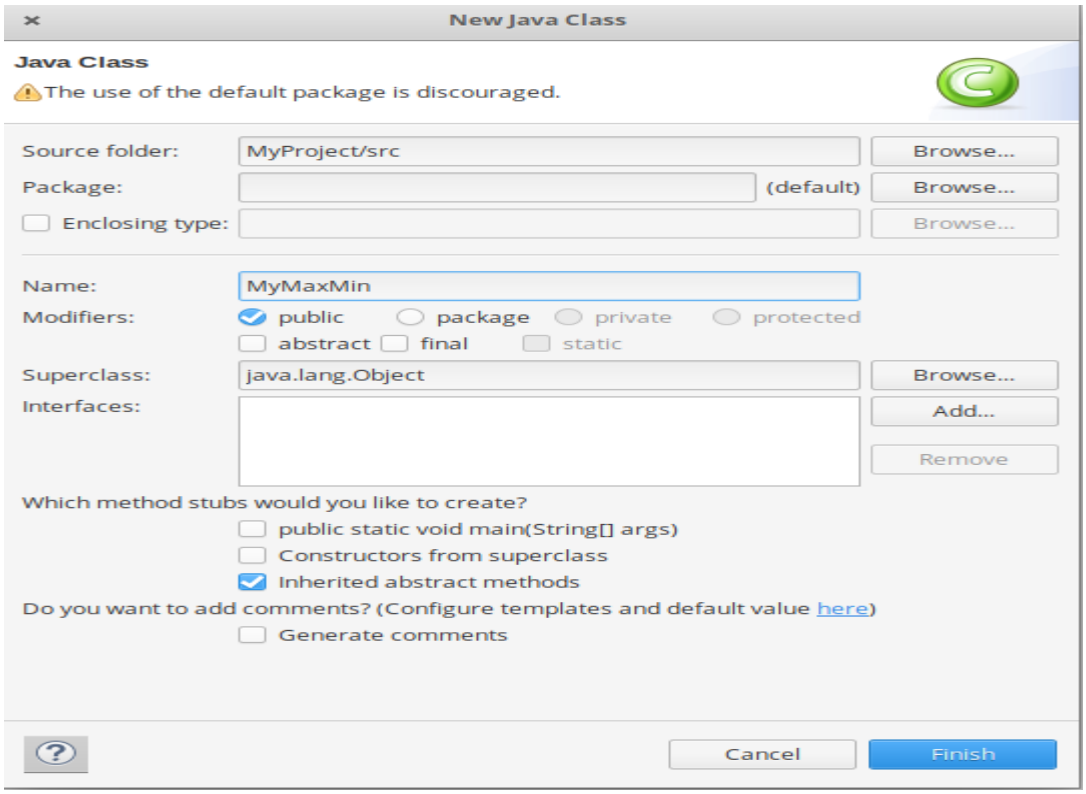
Step 2: Set Up Java Project

Make a project in Eclipse with below steps:

First Open Eclipse -> then, select File -> New -> Java Project -> Name it MyProject -> then, select use an execution environment -> choose, JavaSE-1.8 then, next -> Finish.



In this Project Create Java class with name MyMaxMin -> then, click Finish.



Step 3: Java Source Code

Copy the below source code to this **MyMaxMin** java class

```java
// Required imports for Hadoop MapReduce
import java.io.IOException;
import org.apache.hadoop.fs.Path;
import org.apache.hadoop.io.LongWritable;
import org.apache.hadoop.io.Text;
import org.apache.hadoop.mapreduce.Job;
import org.apache.hadoop.mapreduce.Mapper;
import org.apache.hadoop.mapreduce.Reducer;
import org.apache.hadoop.mapreduce.lib.input.FileInputFormat;
import org.apache.hadoop.mapreduce.lib.output.FileOutputFormat;
import org.apache.hadoop.mapreduce.lib.input.TextInputFormat;
import org.apache.hadoop.mapreduce.lib.output.TextOutputFormat;
import org.apache.hadoop.conf.Configuration;
// Main class
public class MyMaxMin {
   // Mapper class: Extracts max and min temperature from each line
   public static class MaxTemperatureMapper extends Mapper<LongWritable, Text, Text,
Text> {
      // Sentinel value used in dataset to represent missing temperature
      public static final int MISSING = 9999;
      // Map method called for each line in the input file
      @Override
      public void map(LongWritable key, Text value, Context context)
            throws IOException, InterruptedException {
         String line = value.toString(); // Convert line to string
         if (line.length() != 0) {  // Skip empty lines
            // Extract date from line (characters 6 to 14)
            String date = line.substring(6, 14);
            // Extract and trim max and min temperatures
            float temp_Max = Float.parseFloat(line.substring(39, 45).trim());
            float temp_Min = Float.parseFloat(line.substring(47, 53).trim());
            // If max temperature is valid and > 30°C, consider it a hot day
            if (temp_Max != MISSING && temp_Max > 30.0) {
               context.write(new Text("Hot Day: " + date), new
Text(String.valueOf(temp_Max)));
            }
            // If min temperature is valid and < 15°C, consider it a cold day
            if (temp_Min != MISSING && temp_Min < 15.0) {
               context.write(new Text("Cold Day: " + date), new
Text(String.valueOf(temp_Min)));
            }
         }
      }
   }

   // Reducer class: Simply passes through the (key, value) pairs from mapper
   public static class MaxTemperatureReducer extends Reducer<Text, Text, Text, Text> {
      @Override
```

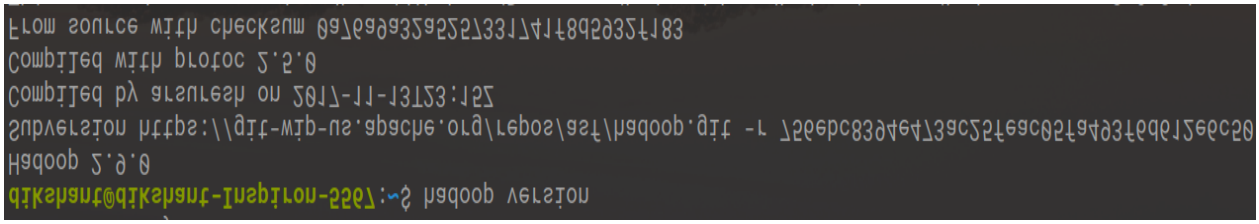
```
    public void reduce(Text key, Iterable<Text> values, Context context)
            throws IOException, InterruptedException {
        // Write each value to the output (usually only one value per key in this case)
        for (Text val : values) {
            context.write(key, val);
        }
    }
}

// Driver method: Configures and starts the MapReduce job
public static void main(String[] args) throws Exception {
    Configuration conf = new Configuration();  // Create Hadoop job configuration
    Job job = Job.getInstance(conf, "Weather Analysis");  // Initialize job with name
    job.setJarByClass(MyMaxMin.class);  // Set main class
    job.setMapperClass(MaxTemperatureMapper.class);  // Set mapper class
    job.setReducerClass(MaxTemperatureReducer.class);  // Set reducer class
    // Set output types for mapper
    job.setMapOutputKeyClass(Text.class);
    job.setMapOutputValueClass(Text.class);
    // Set input/output formats
    job.setInputFormatClass(TextInputFormat.class);
    job.setOutputFormatClass(TextOutputFormat.class);
    // Set input and output file paths from command-line arguments
    FileInputFormat.addInputPath(job, new Path(args[0]));
    FileOutputFormat.setOutputPath(job, new Path(args[1]));
    // Submit job and exit based on completion status
    System.exit(job.waitForCompletion(true) ? 0 : 1);
}}
```

Step 4: Add External JARs

To ensure imported packages work correctly, need to add **external JAR** files in the project..

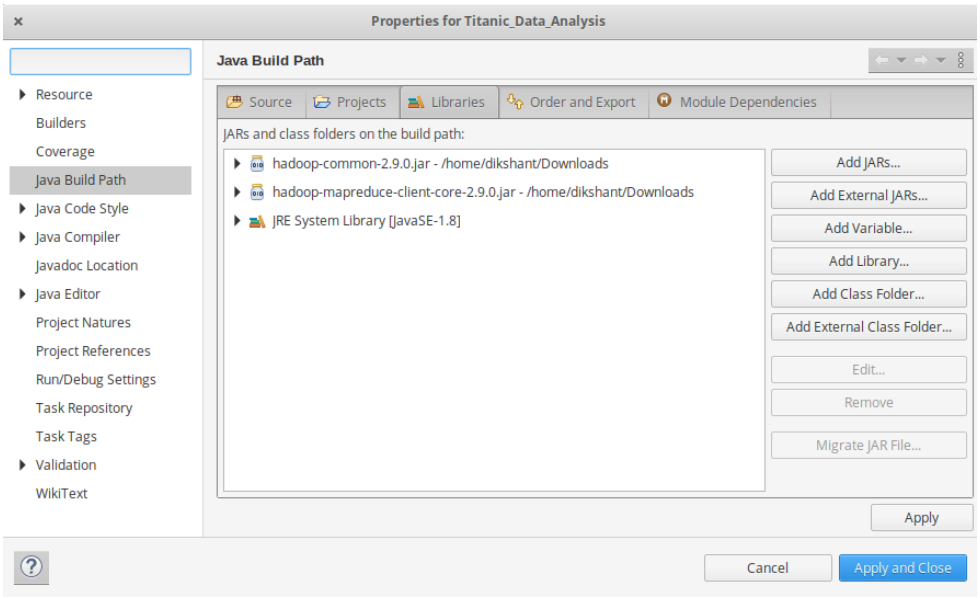
Check Hadoop version with below command:

hadoop version



Now, to add external jars to **MyProject**:

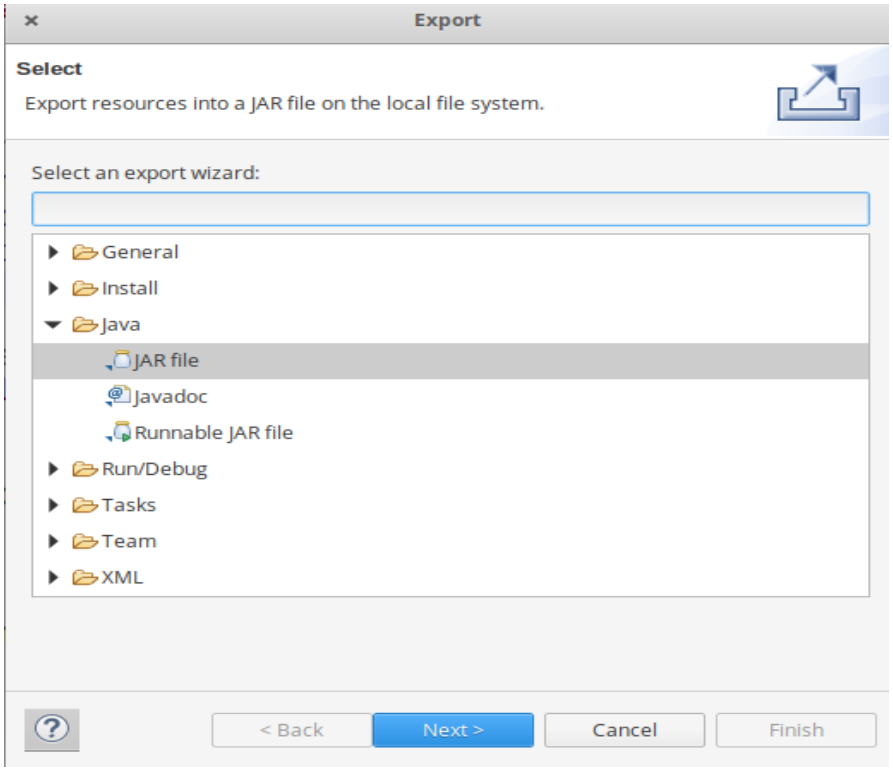
Right Click on **MyProject** -> then, **Build Path** -> Click on, **Configure Build Path** and select **Add External jars** then Add jars from it's download location then click -> **Apply and Close**.



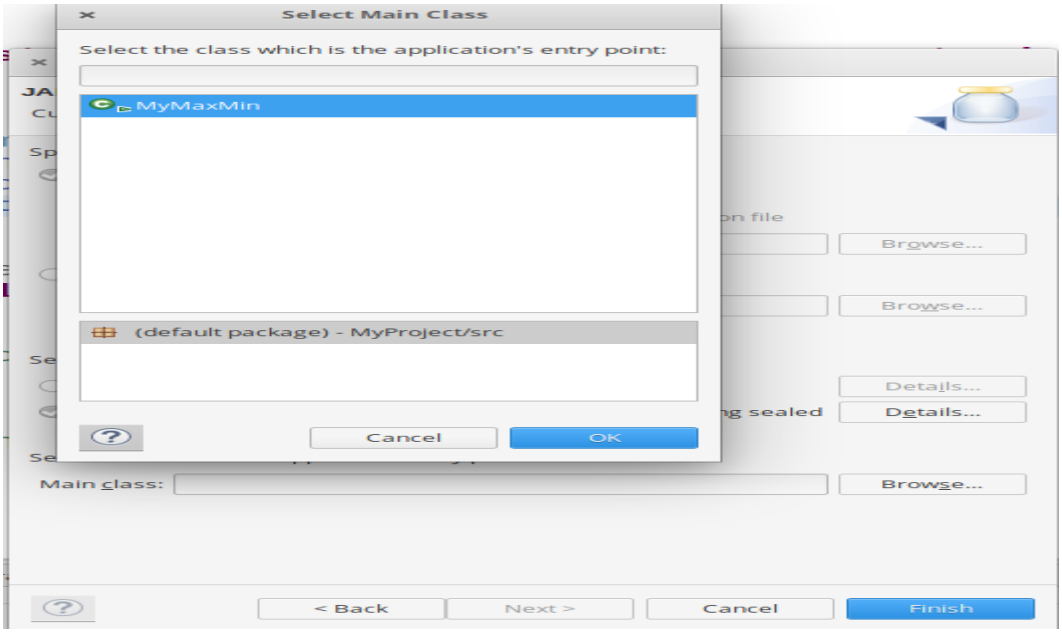
Step 5: Export Project as JAR

Now export the project as jar file.

Right-click on **MyProject** choose **Export** -> go to, **Java** -> **JAR file** -> click, **Next** then, choose your **export destination** then click -> **Next**.



Choose Main Class as **MyMaxMin** by clicking -> **Browse** and then click -> **Finish** -> **Ok**.



Step 6: Start Hadoop Services

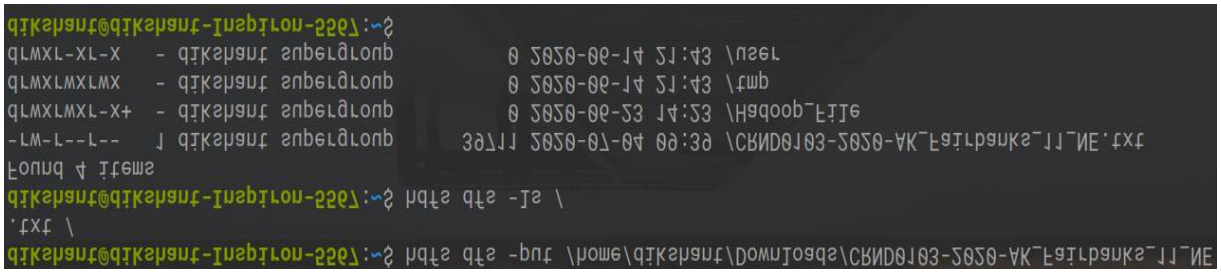
Start HDFS and YARN daemons: *start-dfs.sh ; start-yarn.sh*

Step 7: Move Dataset to HDFS

Command:

hdfs dfs -put /path/to/CRND0103-2020-AK_Fairbanks_11_NE.txt /

To verify: *hdfs dfs -ls /*

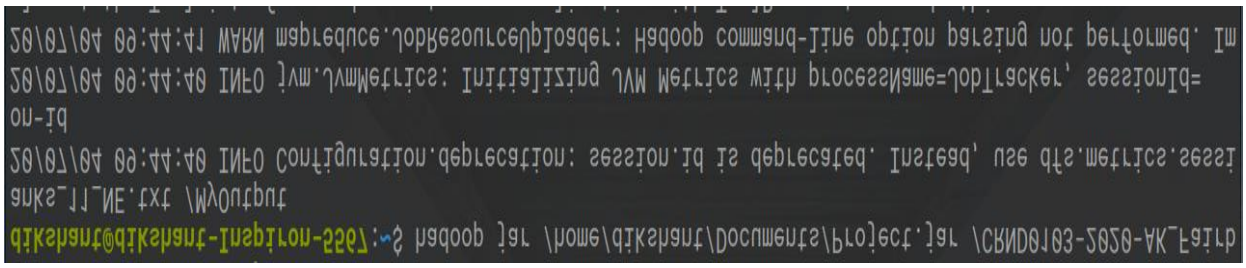


Step 8: Run the MapReduce Job

Now Run your Jar File with below command and produce the output in **MyOutput** File.

Syntax: *hadoop jar /path/to/Project.jar /input_file_in_HDFS /output_directory*

Example: *hadoop jar /home/user/Documents/Project.jar /CRND0103-2020-AK_Fairbanks_11_NE.txt /MyOutput*



Step 9: View Output

After the Map Reduce job completes, you can check the final results through the Hadoop web interface.

Visit:

http://localhost:50070/

Then navigate to: **Utilities** -> Browse the file system -> **/MyOutput** -> **part-r-00000**.

Permission	Owner	Group	Size	Last Modified	Replication	Block Size	Name
-rw-r--r--	dikshant	supergroup	38.78 KB	Jul 04 09:39	1	122.07 MB	CRND0103-2020-AK_Fairbanks_11_NE.txt
drwxrwxr-x+	dikshant	supergroup	0 B	Jun 23 14:23	0	0 B	Hadoop_File
drwxr-xr-x	dikshant	supergroup	0 B	Jul 04 09:44	0	0 B	MyOutput
drwxrwxrwx	dikshant	supergroup	0 B	Jun 14 21:43	0	0 B	tmp
drwxr-xr-x	dikshant	supergroup	0 B	Jun 14 21:43	0	0 B	user

/MyOutput Go!

Show 25 entries Search:

Permission	Owner	Group	Size	Last Modified	Replication	Block Size	Name
-rw-r--r--	dikshant	supergroup	0 B	Jul 04 09:44	1	122.07 MB	_SUCCESS
-rw-r--r--	dikshant	supergroup	3.85 KB	Jul 04 09:44	1	122.07 MB	part-r-00000

Download the result file.

Step 10: Interpret Output

Each line in the output shows:

- **Label:** Hot Day or Cold Day
- **Date:** yyyyMMdd format (e.g., 20200101 = Jan 1, 2020)
- Temperature reading

```
The Day is Cold Day :20200101   -21.8
The Day is Cold Day :20200102   -23.4
The Day is Cold Day :20200103   -25.4
The Day is Cold Day :20200104   -26.8
The Day is Cold Day :20200105   -28.8
The Day is Cold Day :20200106   -30.0
The Day is Cold Day :20200107   -31.4
The Day is Cold Day :20200108   -33.6
The Day is Cold Day :20200109   -26.6
The Day is Cold Day :20200110   -24.3
```

3.2. FILTERING STREAMS USING BLOOM FILTERS

Filtering streams with Bloom filters involves using this space-efficient, probabilistic data structure to quickly check if an item *might* be in a set, preventing expensive lookups, ideal for high-volume data like URLs or user activity, where false positives (saying something's there when it isn't) are acceptable, but false negatives (missing an item) are not, by hashing items to set bits in a bit array and checking for all 1s to confirm potential presence.

How Bloom Filters Work for Streams

- **Initialization**: A bit array (e.g., all zeros) and multiple hash functions are created.
- **Adding Items**: When an item arrives in the stream, it's hashed by each function, producing several indices. The bits at these indices in the array are set to 1.
- **Checking Items**: When a new item arrives, it's hashed, and its corresponding bits are checked.
- If **any** bit is 0, the item is **definitely not** in the set (no false negatives).
- If **all** bits are 1, the item is **possibly** in the set (potential false positive).
- **Filtering**: For a stream, you might add seen items to the filter. New items are checked: if "possibly in set," they're filtered out (or flagged for deeper check); if "definitely not," they pass through.

Key Benefits for Streams

- **Space Efficiency**: Stores only hashed representations, saving significant memory compared to storing full items.
- **Speed**: Hashing and bit checks are very fast, making it ideal for high-throughput data streams.
- **No False Negatives**: Guarantees that if an item *was* added, it will always be detected as present.

Common Use Cases in Streaming

- **Web Browsers**: Filtering malicious URLs (Google Chrome).
- **Recommendation Engines**: Filtering posts users have already seen (Medium, Quora).
- **Databases/Caches**: Reducing disk lookups for non-existent data (BigTable, Cassandra, Redis).
- **Network Routers**: Detecting/filtering unwanted packets.

Limitations (The Trade-off)

- **False Positive:** Can incorrectly flag an item as present (e.g., a URL is safe but the filter says "maybe").
- **No Deletions**: Standard Bloom filters don't support removing items.
- By using Bloom filters, systems can efficiently manage massive data streams, making quick decisions on item membership with minimal memory, accepting a small risk of false positives for huge performance gains.

3.3. UNDERSTANDING HADOOP API FOR MR FRAMEWORK (OLD AND NEW)

In Hadoop, the MapReduce (MR) framework has undergone significant changes in its API structure. The "Old API" refers to versions up to Hadoop 0.20, while the "New API" was introduced in Hadoop 0.21 and remains the standard for Hadoop 2.x and 3.x.

Example of New Mapreduce API is org.apache.hadoop.mapreduce

Example of Old Mapreduce API is org.apache.hadoop.mapred

S.No	Difference	New API	Old API
1	Mapper & Reducer	New API using Mapper and Reducer as Class. So can add a method to an abstract class without breaking old.	In Old API using Mapper and Reducer as Interface.
2	Package	New API is in the ***org.apache.hadoop.mapreduce*** package	Old API can still be found in ***org.apache.hadoop.mapred.***
3	User Code to communicate with Map Reduce System	**use *context* object to communicate with** mapReduce system	***JobConf***, the ***OutputCollector***, and The ***Reporter*** object use for communicate with Map reduce System
4	Control Mapper and Reducer execution	new API allows both mappers and reducers to control the execution flow by ***overriding the run()*** method.	Controlling mappers by writing a ***MapRunnable***, but no equivalent exists for reducers.
5	JOB control	Job control is done through the JOB class in New API	Job Control was done through Job Client (not exists in the new API)
6	Job Configuration	Job Configuration done Through ***Configuration*** class via some of the helper methods	**jobconf** object was used for Job Configuration. Which is extension of Configuration class. java.lang.Object extended by

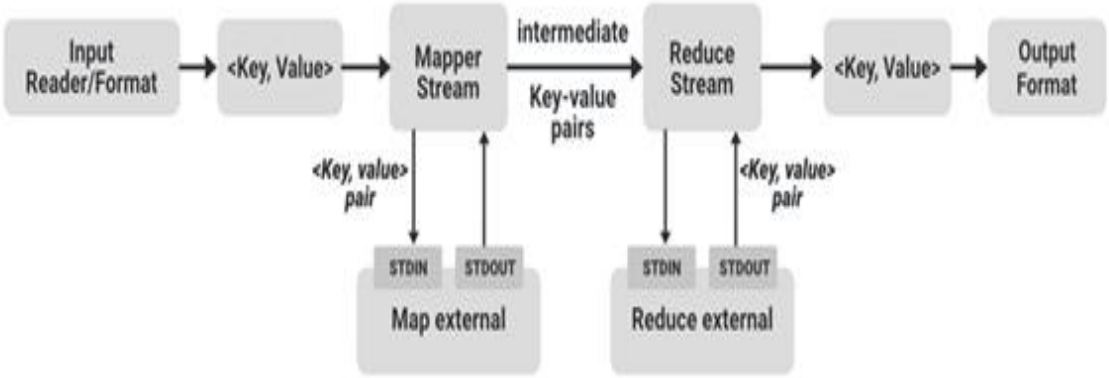
		on Job.	org.apache.hadoop.conf.Configuration extended by org.apache.hadoop.mapred.JobConf
7	OutPut file Name	In the new API map outputs are Named ***part-m-nnnnn***, and reduce outputs are named ***part-r-nnnnn*** (where nnnnn is an integer designating the part number, starting from zero).	in the old API both map and reduce outputs are named ***part-nnnnn***
8	reduce() method passes values	In the new API, the reduce() method passes values as a **java.lang.Iterable**	In the Old API, the reduce() method passes values as a **java.lang.Iterator**

3.4. HADOOP STREAMING

Hadoop Streaming is a utility that allows users to create and run Map-Reduce jobs using any executable or script as the mapper and/or reducer, instead of Java. It enables the use of various programming languages like Python, Ruby, and Perl for processing large datasets. This flexibility makes it easier for non-Java developers to influence Hadoop's distributed computing power for tasks such as log analysis, text processing, and data transformation.

Purpose of Hadoop Streaming: The purpose of Hadoop Streaming, an open-source framework used for efficiently processing large data sets across distributed computing environments, particularly in non-Java programming languages.

How Hadoop Streaming Works?



In the diagram above, the Mapper reads the input data from Input Reader/Format in the form of key-value pair, maps them as per logic written on code, and then passes through the Reduce stream, which performs data aggregation and releases the data to the output.

The MapReduce job demonstrates a flow with an **Input Reader that reads input data and generates a list of key-value pairs**. We can read data in **.csv format**, in delimiter (restrict) format, from a database table, image data (.jpg, .png), audio data etc. The only requirement to read all these types of data is that we have to create a particular input format for that data with these input readers. The input reader contains the complete logic about the data it is reading.

Suppose, we want to read an image then we have to specify the logic in the input reader so that it can read that image data and finally it will generate key-value pairs for that image data. If we are reading an image data then we can generate key-value pair for each pixel where the key will be the location of the pixel and the value will be its color value from (0-255) for a coloured image.

Now this list of key-value pairs is fed to the Map phase and Mapper will work on each of these key-value pair of each pixel and generate some intermediate key-value pairs which are then fed to the Reducer after doing shuffling and sorting then the final output produced by the reducer will be written to the HDFS. These are how a simple Map-Reduce job works.

Hadoop Streaming is an invaluable tool for developers who need to leverage the power of Hadoop without diving deep into Java. Its ability to integrate various programming languages and tools makes it a flexible and powerful option for processing large datasets. Whether you're analyzing logs, processing text data, or running machine learning algorithms, Hadoop Streaming simplifies the process and opens up new possibilities for big data processing.

Features of Hadoop Streaming

Streaming provides several important features:

- Users can execute non-Java-programmed MapReduce jobs on Hadoop clusters. Supported languages include Python, Perl, and C++.
- Hadoop Streaming monitors the progress of jobs and provides logs of a job's entire execution for analysis.
- Hadoop Streaming works on the MapReduce paradigm, so it supports scalability, flexibility, and security/authentication.
- Hadoop Streaming jobs are quick to develop and don't require much programming (except for executables).

3.5. BASIC PROGRAMS OF HADOOP MAP REDUCE TYPES AND FORMATS

Hadoop MapReduce is a distributed processing framework that operates exclusively on `<key, value>` pairs. To process diverse data types, it uses specific **Input Formats** to define how files are split and read into records, and **Output Formats** to define how results are saved.

i). MapReduce Data Types

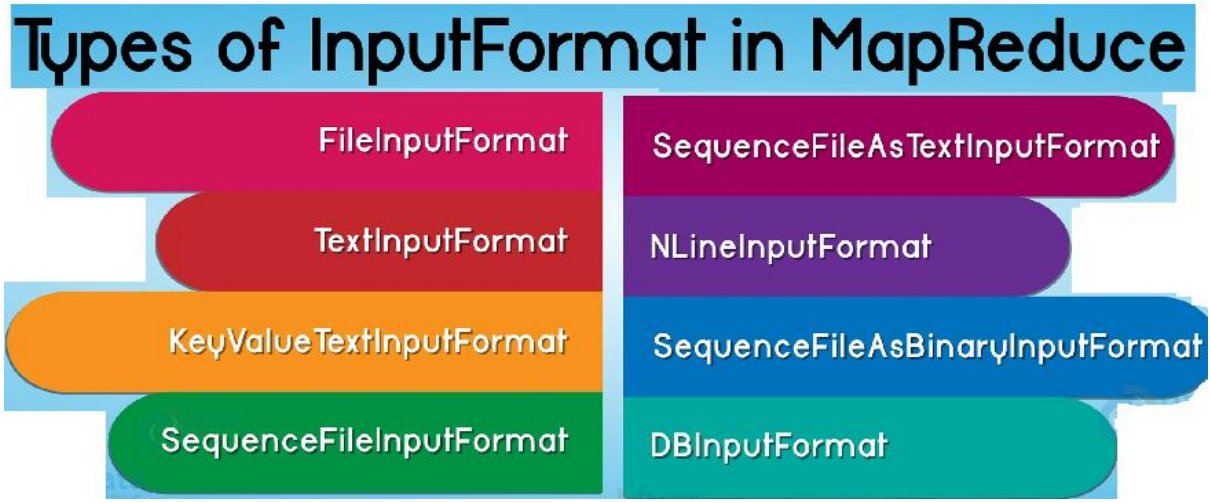
In Hadoop, keys and values must be serializable to move across the network. All data types used in MapReduce must implement the Writable Interface.

- **Keys:** Must implement `WritableComparable` to facilitate sorting.
- **Values:** Must implement `Writable`.
- **Common Classes:** `Text` (like String), `IntWritable`, `LongWritable`, `FloatWritable`, and `BooleanWritable`.

ii). Input Formats

The `InputFormat` class defines how input files are split into `InputSplits` and converted into records by a `RecordReader`.

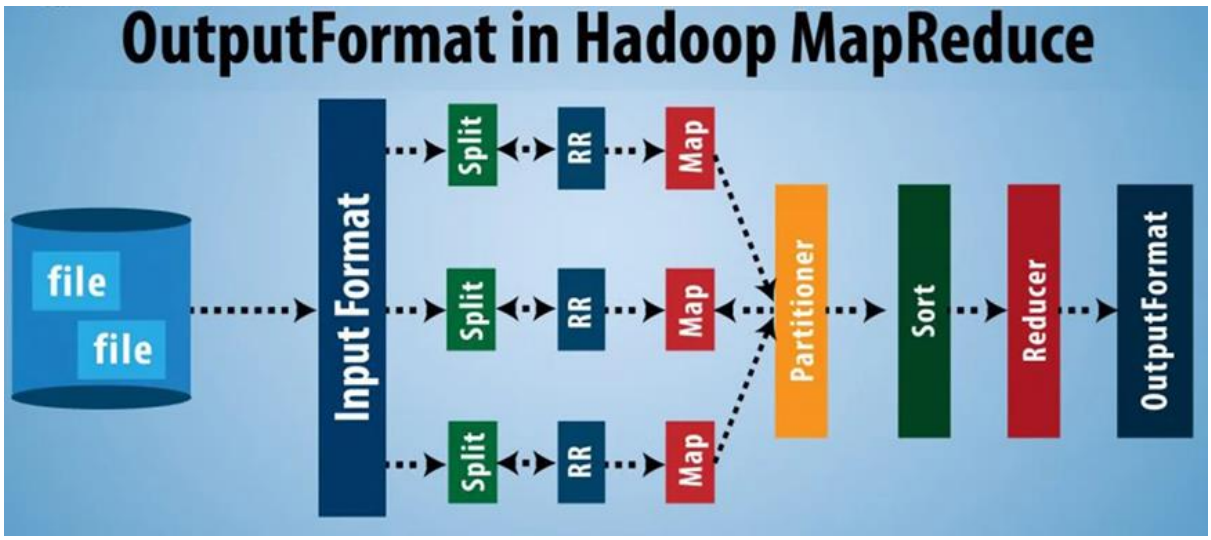
- **TextInputFormat (Default):** Each line of the file is a record. The key is the byte offset of the line (LongWritable), and the value is the line's content (Text).
- **KeyValueTextInputFormat:** Similar to `TextInputFormat`, but it splits each line into a key and a value using a delimiter (usually a tab `\t`).
- **NLineInputFormat:** Ensures each mapper receives exactly *N* lines of input, regardless of split size.
- **SequenceFileInputFormat:** Reads SequenceFiles, which are binary files storing key-value pairs efficiently.
- **DBInputFormat:** Allows MapReduce to read data directly from a relational database using JDBC.



iii). Output Formats

The `OutputFormat` class specifies where and how the final results are written to HDFS.

- **TextOutputFormat (Default):** Writes records as plain text lines. Keys and values are separated by tabs.
- **SequenceFileOutputFormat:** Writes output in binary sequence file format, useful for passing data between consecutive MapReduce jobs.
- **MultipleOutputs:** Allows a single MapReduce job to write to multiple files with different names or formats based on the data content.
- **LazyOutputFormat:** A wrapper that prevents the creation of empty output files if no records are emitted by a reducer.



iv). Basic MapReduce Program Structure

A standard Hadoop MapReduce job involves three main components:

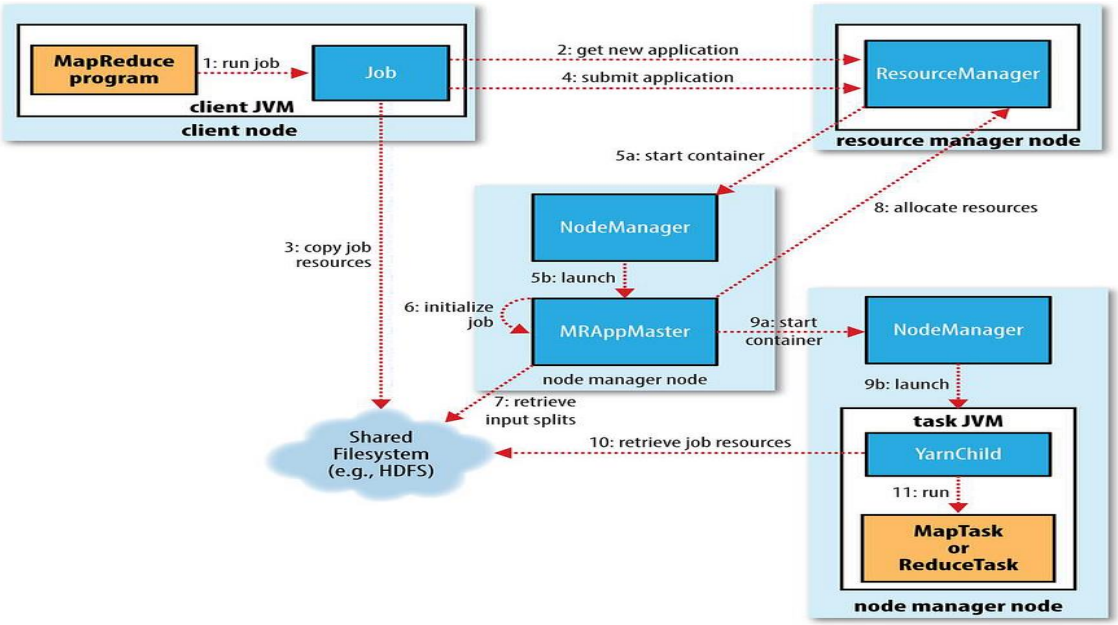
- **Mapper:** Processes input records and generates intermediate key-value pairs.

- **Reducer:** Aggregates intermediate pairs based on common keys to produce the final output.
- **Driver:** Configures the job (sets the Mapper, Reducer, InputFormat, and OutputFormat) and submits it to the cluster.

3.6. ANATOMY OF A MAPREDUCE JOB RUN

The entire MapReduce job execution involves several steps beyond just mapping and reducing:

- Job Submission
- Job Initialization
- Task Assignment
- Task Execution
- Progress and Status Updates
- Job Completion



i).Job Submission: The client submits the MapReduce job, which includes the application code, configuration, and input/output paths.

- The `submit()` method on Job creates an internal `JobSubmitter` instance and calls `submitJobInternal()` on it.
- Having submitted the job, `waitForCompletion` polls the job's progress once per second and reports the progress to the console if it has changed since the last report.
- When the job completes successfully, the job counters are displayed otherwise, the error that caused the job to fail is logged to the console.

The job submission process implemented by `JobSubmitter` does the following:

- Asks the resource manager for a new application ID, used for the MapReduce job ID.
- Checks the output specification of the job For example, if the output directory has not been specified or it already exists, the job is not submitted and an error is thrown to the MapReduce program.
- Computes the input splits for the job If the splits cannot be computed (because the input paths don't exist, for example), the job is not submitted and an error is thrown to the MapReduce program.
- Copies the resources needed to run the job, including the job JAR file, the configuration file, and the computed input splits, to the shared file system in a directory named after the job ID.
- Submits the job by calling submitApplication() on the resource manager.

ii). Job Initialization: The YARN (Yet Another Resource Negotiator) Resource Manager allocates resources and launches the Application Master, which coordinates the tasks within the job.

- When the resource manager receives a call to its submitApplication() method, it hands off the request to the YARN scheduler.
- The scheduler allocates a container, and the resource manager then launches the application master's process there, under the node manager's management.
- The application master for MapReduce jobs is a Java application whose main class is MRAppMaster.
- It initializes the job by creating a number of bookkeeping objects to keep track of the job's progress, as it will receive progress and completion reports from the tasks.
- It retrieves the input splits computed in the client from the shared filesystem.
- It then creates a map task object for each split, as well as a number of reduce task objects determined by the mapreduce.job.reduces property (set by the setNumReduceTasks() method on Job).

iii). Task Assignment: The Application Master requests containers for map and reduce tasks from the Resource Manager. Map tasks are often prioritized for data locality (running on nodes where the input data resides).

- If the job does not qualify for running as an user task, then the application master requests containers for all the map and reduce tasks in the job from the resource manager.

- Requests for map tasks are made first and with a higher priority than those for reduce tasks, since all the map tasks must complete before the sort phase of the reduce can start.
- Requests for reduce tasks are not made until 5% of map tasks have completed.

iv).Task Execution: Node Managers launch and monitor the containers where Map and Reduce tasks execute. Each task is run within a dedicated Java Virtual Machine (JVM) for isolation.

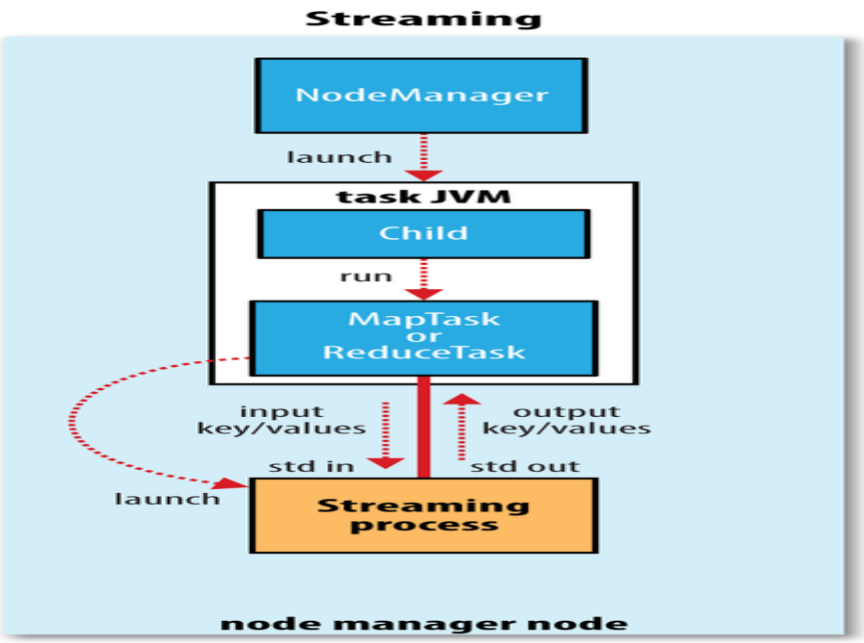
- Once a task has been assigned resources for a container on a particular node by the resource manager's scheduler, the application master starts the container by contacting the node manager.
- The task is executed by a Java application whose main class is YarnChild. Before it can run the task, it localizes the resources that the task needs, including the job configuration and JAR file, and any files from the distributed cache.
- Finally, it runs the map or reduce task.

v).Progress and Status Updates: Tasks periodically report their progress and status to the Application Master, which in turn aggregates the information and reports it to the client.

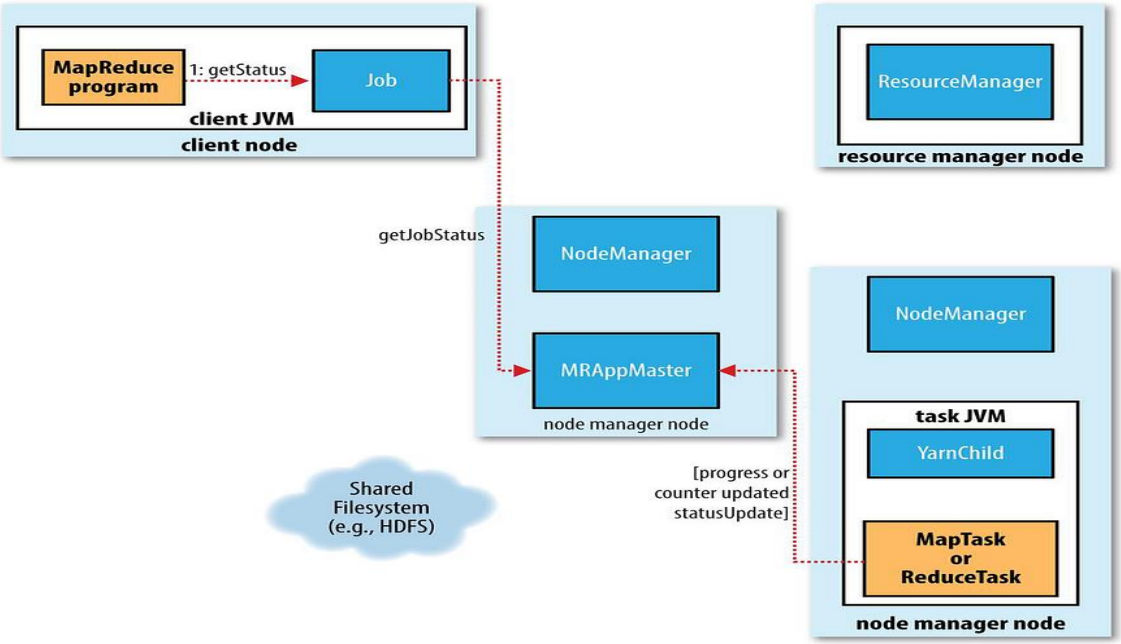
- MapReduce jobs are long running batch jobs, taking anything from tens of seconds to hours to run.
- A job and each of its tasks have a status, which includes such things as the state of the job or task (e g running, successfully completed, failed), the progress of maps and reduces, the values of the job's counters, and a status message or description (which may be set by user code).
- When a task is running, it keeps track of its progress (i e the proportion of task is completed).
- For map tasks, this is the proportion of the input that has been processed.
- For reduce tasks, it's a little more complex, but the system can still estimate the proportion of the reduce input processed.

It does this by dividing the total progress into three parts, corresponding to the three phases of the shuffle.

- As the map or reduce task runs, the child process communicates with its parent application master through the umbilical interface.
- The task reports its progress and status (including counters) back to its application master, which has an aggregate view of the job, every three seconds over the umbilical interface.



- Streaming runs special map and reduce tasks for the purpose of launching the user supplied executable and communicating with it.
- The Streaming task communicates with the process (which may be written in any language) using standard input and output streams.
- During execution of the task, the Java process passes input key value pairs to the external process, which runs it through the user defined map or reduce function and passes the output key value pairs back to the Java process.
- From the node manager's point of view, it is as if the child process ran the map or reduce code itself.



How status updates are propagated through the Map-Reduce System

- The resource manager web UI displays all the running applications with links to the web UIs of their respective application masters, each of which displays further details on the MapReduce job, including its progress.
- During the course of the job, the client receives the latest status by polling the application master every second (the interval is set via mapreduce.client.progressmonitor.pollinterval).

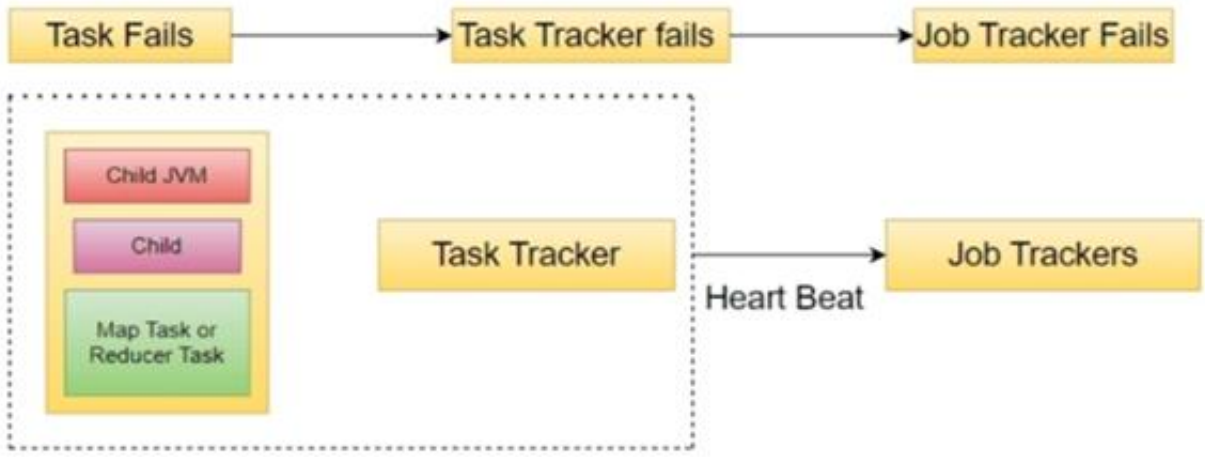
vi).Job Completion: Once all tasks are complete, the Application Master notifies the client, cleans up resources, and archives job information to a history server.

- When the application master receives a notification that the last task for a job is complete, it changes the status for the job to Successful.
- Then, when the Job polls for status, it learns that the job has completed successfully, so it prints a message to tell the user and then returns from the waitForCompletion() .
- Finally, on job completion, the application master and the task containers clean up their working state and the OutputCommitter's commitJob () method is called.
- Job information is archived by the job history server to enable later interrogation by users if desired.

3.7. FAILURES

There are generally 3 types of failures in MapReduce.

- Task Failure
- TaskTracker Failure
- JobTracker Failure.



Failures in MapReduce

i).Task Failure: In Hadoop, task failure is similar to an employee making a mistake while doing a task. Consider you are working on a large project that has been broken down into smaller jobs and assigned to different employees in your team. If one of the team members fails to do their task correctly, the entire project may be compromised. Similarly, in Hadoop, if a job fails due to a mistake or issue, it could affect overall data processing, causing delays or faults in the final result.

Reasons for Task Failure:

- **Limited memory:** A task can fail if it runs out of memory while processing data.
- **Failures of disk:** If the disk that stores data or intermediate results fails, tasks that depend on that data may fail.
- **Issues with software or hardware:** Bugs, mistakes, or faults in software or hardware components can cause task failures.

How to Overcome Task Failure

- ✓ **Increase memory allocation:** Assign extra memory to jobs to ensure they have the resources to process the data.
- ✓ **Implement fault tolerance mechanisms:** Using data replication and checkpointing techniques to defend against disc failures and retrieve lost data.
- ✓ **Regularly update software and hardware:** Keep the Hadoop framework and supporting hardware up to date to fix bugs, errors, and performance issues that can lead to task failures.

ii). TaskTracker Failure: A **TaskTracker** in Hadoop is similar to an employee responsible for executing certain tasks in a large project. If a TaskTracker fails, it signifies a problem occurred while an employee worked on their assignment. This can interrupt the entire project, much as when a team member makes a mistake or encounters difficulties with their task, producing delays or problems with the overall project's completion. To avoid TaskTracker failures, ensure the TaskTracker's hardware and software are in excellent working order and have the resources they need to do their jobs successfully.

Reasons for TaskTracker Failure

- **Hardware issues:** Just as your computer's parts can break or stop working properly, the TaskTracker's hardware (such as the processor, memory, or disc) might fail or stop operating properly. This may prohibit it from carrying out its duties.

- **Software problems or errors:** The software operating on the TaskTracker may contain bugs or errors that cause it to cease working properly. It's similar to when an app on your phone fails and stops working properly.
- **Overload or resource exhaustion:** It may struggle to keep up if the TaskTracker becomes overburdened with too many tasks or runs out of resources such as memory or processing power. It's comparable to being overburdened with too many duties or running out of storage space on your gadget.

How to Overcome TaskTracker Failure

- ✓ **Update software and hardware on a regular basis:** Keep the Hadoop framework and associated hardware up to date to correct bugs, errors, and performance issues that might lead to task failures.
- ✓ **Upgrade or replace hardware:** If TaskTracker's hardware is outdated or insufficiently powerful, try upgrading or replacing it with more powerful components. It's equivalent to purchasing a new, upgraded computer to handle jobs more efficiently.
- ✓ **Restart or reinstall the program:** If the TaskTracker software is causing problems, a simple restart or reinstall may be all that is required. It's the same as restarting or reinstalling an app to make it work correctly again.

iii).JobTracker Failure: A **JobTracker** in Hadoop is similar to a supervisor or manager that oversees the entire project and assigns tasks to TaskTrackers (employees). If a JobTracker fails, it signifies the supervisor is experiencing a problem or has stopped working properly. This can interrupt the overall project's coordination and development, much as when a supervisor is unable to assign assignments or oversee their completion. To avoid JobTracker failures, it is critical to maintain the JobTracker's hardware and software, ensure adequate resources, and fix any issues or malfunctions as soon as possible to keep the project going smoothly.

Reasons for JobTracker Failure:

- **Database connectivity:** The JobTracker stores job metadata and state information in a backend database (usually Apache Derby or MySQL). JobTracker failures can occur if there are database connectivity issues, such as network problems or database server failures.
- **Security problems:** JobTracker failures can be caused by security issues such as authentication or authorization failures, incorrectly configured security settings or key distribution and management issues.

How to Overcome JobTracker Failure

- ✓ **Avoiding Database Connectivity:** To avoid database connectivity failures in the JobTracker, ensure optimized database configuration, robust network connections, and high availability techniques are implemented. Retrying connections, monitoring, and backups are all useful.
- ✓ **To overcome security-related problems:** implement strong authentication and authorization, enable SSL/TLS for secure communication, keep software updated with security patches, follow key management best practices, conduct security audits, and seek expert guidance for vulnerability mitigation and compliance with security standards.

Impact of Failures:

- Completed Tasks: Completed map tasks are re-executed because their output is stored locally on the failed worker's disk.
- Running Tasks: Running map and reduce tasks are reset to their initial state and rescheduled on other available nodes.
- Rescheduling: The MapReduce framework automatically reschedules failed tasks, ensuring that the job eventually completes, even with failures.

3.8. MAP REDUCE DRIVER CODE

The Driver code runs on the client machine and is responsible for building the configuration of the job and submitting it to the Hadoop Cluster. The Driver code will contain the main() method that accepts arguments from the command line. Some of the common libraries that are included for the Driver class :

```
import org.apache.hadoop.fs.Path;
import org.apache.hadoop.io.*;
import org.apache.hadoop.mapred.*;
```

In most cases, the command line parameters passed to the Driver program are the paths to the directory where containing the input files and the path to the output directory. Both these path locations are from the HDFS. The output location should not be present before running the program as it is created after the execution of the program. If the output location already exists the program will exit with an error.

The next step the Driver program should do is to configure the Job that needs to be submitted to the cluster. To do this we create an object of type JobConf and pass the name of

the Driver class. The JobConf class allows you to configure the different properties for the Mapper, Combiner, Partitioner, Reducer, InputFormat and OutputFormat.

Sample

```
public class MyDriver{
public static void main(String[] args) throws Exception
{ // Create the JobConf object
JobConf conf = new JobConf(MyDriver.class);
// Set the name of the Job
conf.setJobName(―SampleJobName‖);
// Set the output Key type for the Mapper
conf.setMapOutputKeyClass(Text.class);
// Set the output Value type for the Mapper
conf.setMapOutputValueClass(IntWritable.class);
// Set the output Key type for the Reducer
conf.setOutputKeyClass(Text.class);
// Set the output Value type for the Reducer
conf.setOutputValueClass(IntWritable.class);

// Set the Mapper Class
conf.setMapperClass(MyMapper.class);
// Set the Reducer Class
conf.setReducerClass(Reducer.class);
// Set the format of the input that will be provided to the program
conf.setInputFormat(TextInputFormat.class);
// Set the format of the output for the program
conf.setOutputFormat(TextOutputFormat.class);
// Set the location from where the Mapper will read the input
FileInputFormat.setInputPaths(conf, new Path(args[0]));
// Set the location where the Reducer will write the output
FileOutputFormat.setOutputPath(conf, new Path(args[1]));
// Run the job on the cluster JobClient.runJob(conf);
}
}
```

3.9. MAPPER CODE

The **mapper** is the first phase of the MapReduce programming model, responsible for processing raw input data in parallel and transforming it into intermediate **key-value pairs**. It is a user-defined function or class that applies custom logic to each record of the input data.

Key Functions and Workflow of the Mapper

- **Input Processing:** The raw input data (typically stored in the Hadoop Distributed File System, or HDFS) is divided into logical chunks called **input splits**. Each input split is assigned to a separate mapper task, enabling parallel processing.
- **Record Conversion:** A `RecordReader` associated with the `InputFormat` converts the raw input of an input split into a series of key-value pairs, which are then fed to the mapper. By default, for text files, the key is the byte offset of the line, and the value is the line of text itself.
- **Map Function:** The mapper's core logic resides in its `map()` method, which processes each input key-value pair and generates zero or more new, intermediate key-value pairs. The types of the intermediate key-value pairs can be completely different from the input pairs.
- **Intermediate Output:** The mapper's output, known as intermediate data, is temporarily stored on the local disk of the machine running the mapper task, not in HDFS.
- **Preparation for Reducer:** After all mappers complete their tasks, the intermediate key-value pairs undergo an automatic **shuffle and sort** phase by the MapReduce framework, which groups all values associated with the same key together and sorts them by key. This sorted, grouped data is then sent to the reducer phase for final aggregation and processing.

The Mapper code reads the input files as <Key,Value> pairs and emits key value pairs. The Mapper class extends MapReduceBase and implements the Mapper interface. The Mapper interface expects four generics, which define the types of the input and output key/value pairs. The first two parameters define the input key and value types, the second two define the output key and value types.

Some of the common libraries that are included for the Mapper class :

```
public class MyMapper extends MapReduceBase implements
Mapper<LongWritable, Text, Text, IntWritable>{
```

```
public void map(LongWritable key, Text value, OutputCollector<Text, IntWritable>
 output, Reporter reporter) throws IOException { output.collect(key,value);
}
}
}
```

The map() function accepts the key, value, OutputCollector and an Reporter object. The OutputCollector is resposible for writing the intermediate data generated by the Mapper.

3.10 REDUCER CODE

In MapReduce, the Reducer is the second phase that aggregates the intermediate key-value pairs from the Mapper into a smaller, consolidated final output, performing tasks like summing, averaging, or filtering data for specific keys, and writing the results to HDFS. It receives grouped data (e.g., `(Key, [Value1, Value2, ...])`) after Shuffling and Sorting, then processes these lists of values to generate the final results.

How the Reducer Works:

- **Input:** Takes the output from Mappers, which are intermediate `(Key, Value)` pairs.
- **Shuffle & Sort:** Before Reducers run, the framework sorts and shuffles data, grouping all values associated with the same key together (e.g., `(Key, List<Values>)`).
- **Aggregation:** The Reducer's `reduce()` method is called once for each unique key, iterating over the list of values to perform calculations (like summing salaries for a job title).
- **Output:** Produces final `(Key, Value)` pairs, which are then written to the HDFS.

Key Functions:

- **Consolidation:** Reduces many intermediate values to fewer, aggregated results.
- **Calculation:** Performs final sums, counts, averages, or other complex logic.
- **Filtering/Combining:** Filters or combines data into a more meaningful final dataset.

Reducer

The Reducer code reads the outputs generated by the different mappers as <Key,Value> pairs and emits key value pairs. The Reducer class extends MapReduceBase and implements the Reducer interface. The Reducer interface expects four generics, which define the types of the input and output key/value pairs. The first two parameters define the intermediate key and

value types, the second two define the final output key and value types. The keys are WritableComparables, the values are Writables.

Some of the common libraries that are included for the Reducer class :

```
import java.io.IOException; import java.util.*;
import org.apache.hadoop.io.*;
import org.apache.hadoop.mapred.*;
```

Sample

```
public class MyReducer extends MapReduceBase implements
Reducer<Text,IntWritable,Text,IntWritable>
{ @Override
public void reduce(Text key, Iterator<IntWritable> values, OutputCollector<Text,
IntWritable> output, Reporter reporter) throws
IOException{ output.collect(key,value);
}}}
```

The reduce() function accepts the key, an iterator , OutputCollector and an Reporter object. The OutputCollector is resposible for writing the final output result.

3.11. MAP REDUCE RECORDER CODE

RecordReader is the critical component that bridges the gap between raw data blocks and the Mapper. Its primary role is to convert the byte-oriented view of input data, provided by an InputSplit , into a record-oriented view (key-value pairs) that the Mapper can understand and process.

Core Functions

- **Data Translation:** It reads raw data from HDFS blocks and parses it into logical records.
- **Key-Value Pair Generation:** It transforms each record into a specific (Key, Value) pair. For example, in a standard text file, the key might be the byte offset of the line and the value would be the text of the line itself.
- **Boundary Management:** It ensures that even if a record (like a line of text) is split across two HDFS blocks, the complete record is sent to a single Mapper. It does this by reading slightly beyond its designated split boundary to complete the final record.

The Workflow

- ✓ **Creation:** For every `InputSplit`, the `InputFormat` class creates a corresponding `RecordReader` instance.
- ✓ **Initialization:** The framework calls `initialize()` to set up the reader with the split and context.
- ✓ **Iterative Reading:** The framework repeatedly calls `nextKeyValue()` to fetch the next record. If it returns `true`, the framework retrieves the key and value via `getCurrentKey()` and `getCurrentValue()` and passes them to the Mapper's `map()` method.
- ✓ **Completion:** Once the entire split is consumed, `nextKeyValue()` returns `false`, and the reader is closed.

Record Reader: This is the first phase of MapReduce where the Record Reader reads every line from the input text file as text and yields output as key-value pairs.

Input − Line by line text from the input file.

Output − Forms the key-value pairs.

The following is the set of expected key-value pairs.

<1, What do you mean by Object>

<2, What do you know about Java>

<3, What is Java Virtual Machine>

<4, How Java enabled High Performance> Map Phase

The Map phase takes input from the Record Reader, processes it, and produces the output as another set of key-value pairs.

Input − The following key-value pair is the input taken from the Record Reader.

<1, What do you mean by Object>

<2, What do you know about Java>

<3, What is Java Virtual Machine>

<4, How Java enabled High Performance>

The Map phase reads each key-value pair, divides each word from the value using StringTokenizer, treats each word as key and the count of that word as value. The following code snippet shows the Mapper class and the map function.

```
public static class TokenizerMapper extends Mapper<Object, Text, Text, IntWritable> {
private final static IntWritable one = new IntWritable(1); private Text word = new Text();
public void map(Object key, Text value, Context context) throws IOException, InterruptedException
{
StringTokenizer itr = new StringTokenizer(value.toString()); while (itr.hasMoreTokens())
{
word.set(itr.nextToken()); context.write(word, one);
}
}
}
```

Output − The expected output is as follows −

<What,1> <do,1> <you,1> <mean,1> <by,1> <Object,1>

<What,1> <do,1> <you,1> <know,1> <about,1> <Java,1>

<What,1> <is,1> <Java,1> <Virtual,1> <Machine,1>

<How,1> <Java,1> <enabled,1> <High,1> <Performance,1>

3.12. COMBINER

A Combiner acts as a "mini-reducer" on the mapper's output. Its primary role is to perform local aggregation of intermediate key-value pairs on the mapper's node before they are sent to the reducers. This can significantly reduce the amount of data transferred across the network during the shuffle and sort phase, improving job performance. Combiners are optional and are only effective when the reduction operation is commutative and associative (the order of operations doesn't affect the result).

Purpose: The combiner acts as a mini-reducer, operating on the output of individual mappers to partially aggregate data with the same key.

Benefits: It reduces the amount of data sent to reducers, leading to faster job completion and reduced network congestion.

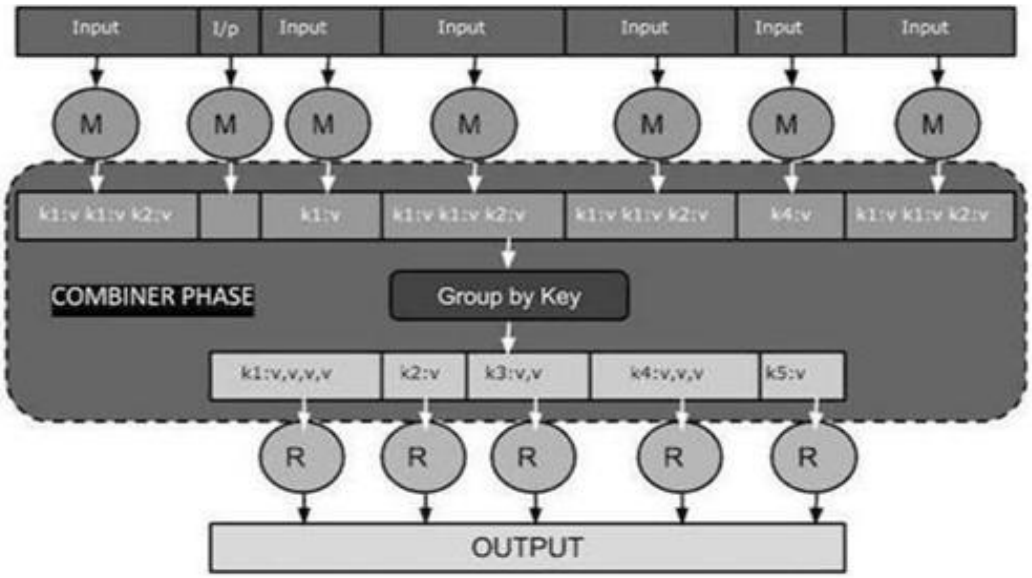
Limitations:

- Resource Usage: Combiners can increase CPU and memory usage on the mapper nodes.

- Data Inconsistencies: If not implemented correctly, combiners can lead to data inconsistencies because they perform partial aggregations.
- Not Guaranteed: The execution of combiners is not guaranteed, and the framework may skip them under certain conditions.

The main function of a Combiner is to summarize the map output records with the same key. The output (key-value collection) of the combiner will be sent over the network to the actual Reducer task as input. The Combiner class is used in between the Map class and the Reduce class to reduce the volume of data transfer between Map and Reduce. Usually, the output of the map task is large and the data transferred to the reduce task is high.

The following MapReduce task diagram shows the COMBINER PHASE.



- ✓ A combiner does not have a predefined interface and it must implement the Reducer interface's reduce() method.
- ✓ A combiner operates on each map output key. It must have the same output key-value types as the Reducer class.
- ✓ A combiner can produce summary information from a large dataset because it replaces the original Map output.
- ✓ Although, Combiner is optional yet it helps segregating data into multiple groups for Reduce phase, which makes it easier to process.

MapReduce Combiner Implementation

The following example provides a theoretical idea about combiners. Let us assume we have the following input text file named input.txt for MapReduce.

- What do you mean by Object
- What do you know about Java
- What is Java Virtual Machine
- How Java enabled High Performance

Combiner Phase: The Combiner phase takes each key-value pair from the Map phase, processes it, and produces the output as key-value collection pairs.

Input − The following key-value pair is the input taken from the Map phase.

<What,1> <do,1> <you,1> <mean,1> <by,1> <Object,1>

<What,1> <do,1> <you,1> <know,1> <about,1> <Java,1>

<What,1> <is,1> <Java,1> <Virtual,1> <Machine,1>

<How,1> <Java,1> <enabled,1> <High,1> <Performance,1>

The Combiner phase reads each key-value pair, combines the common words as key and values as collection. Usually, the code and operation for a Combiner is similar to that of a Reducer. Following is the code snippet for Mapper, Combiner and Reducer class declaration.

```
job.setMapperClass(TokenizerMapper.class);
job.setCombinerClass(IntSumReducer.class);
job.setReducerClass(IntSumReducer.class);
```

Output − The expected output is as follows –

<What,1,1,1> <do,1,1> <you,1,1> <mean,1> <by,1>

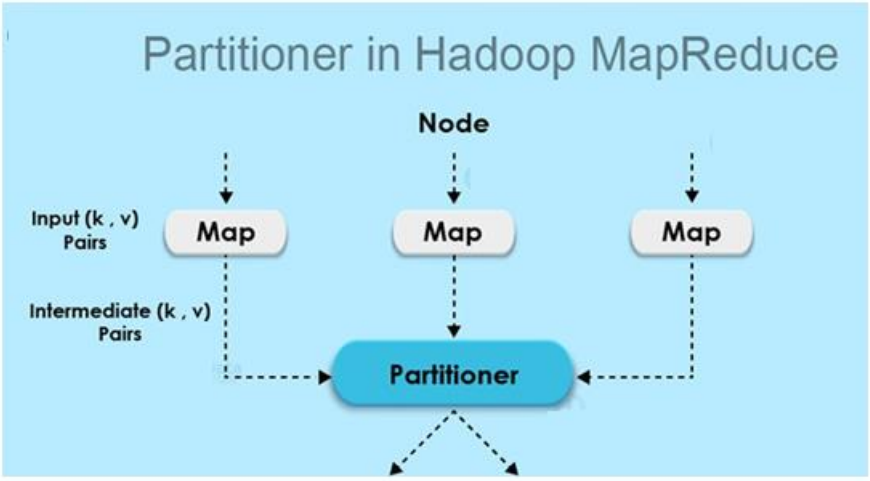
<Object,1> <know,1> <about,1> <Java,1,1,1> <is,1>

<Virtual,1> <Machine,1>

<How,1> <enabled,1> <High,1> <Performance,1>

3.13. PARTITIONER

The Partitioner in MapReduce controls the partitioning of the key of the intermediate mapper output. By hash function, key (or a subset of the key) is used to derive the partition. A total number of partitions depends on the number of reduce task.



Hadoop Partitioner – Internals of Map Reduce Partitioner

Hadoop Partitioner: Partitioning of the keys of the intermediate map output is controlled by the Partitioner. By hash function, key (or a subset of the key) is used to derive the partition. According to the key-value each mapper output is partitioned and records having the same key value go into the same partition (within each mapper), and then each partition is sent to a reducer. Partition class determines which partition a given (key, value) pair will go. Partition phase takes place after map phase and before reduce phase.

Need of Hadoop MapReduce Partitioner: Map Reduce job takes an input data set and produces the list of the key-value pair which is the result of map phase in which input data is split and each task processes the split and each map, output the list of key-value pairs. Then, the output from the map phase is sent to reduce task which processes the user-defined reduce function on map outputs. But before reduce phase, partitioning of the map output take place on the basis of the key and sorted.

This partitioning specifies that all the values for each key are grouped together and make sure that all the values of a single key go to the same reducer, thus allows even distribution of the map output over the reducer. Partitioner in Hadoop MapReduce redirects the mapper output to the reducer by determining which reducer is responsible for the particular key.

Default MapReduce Partitioner: The Default Hadoop partitioner in Hadoop MapReduce is Hash Partitioner which computes a hash value for the key and assigns the partition based on this result.

How many Partitioners are there in Hadoop: The total number of Partitioners that run in Hadoop is equal to the number of reducers i.e. Partitioner will divide the data according to the number of reducers which is set by JobConf.setNumReduceTasks() method. Thus, the data

from single partitioner is processed by a single reducer. And partitioner is created only when there are multiple reducers.

2MARKS QUESTIONS

1. What is Map Reduce?
2. What are the advantages of Map-Reduce?
3. What are the benefits of Streaming?
4. Mention any two difference between new API and Old API of Hadoop?
5. What is the purpose of Hadoop Streaming?
6. What are the steps to run a Map-Reduce Job? (Anatomy of MR Jobs)
7. Define Failure?
8. How many types of failures in MR Jobs?
9. How to overcome Task failure?
10. What are the reasons for Job Tracker failure?
11. Define mapper?
12. Define Reducer?
13. What are the core functions of Map-reduce recorder?
14. What is the work of Combiner?
15. What is the practitioner?

Prepared By

Dr.K Venkata Nagendra,

M.Tech, Ph.D, PDF.